



Centro Para la
Instrumentación Astronómica



Universidad
de Concepción

CePIA

Sistema de Monitoreo de Temperatura para Radiómetro

Proyecto WVRs

Preparado por:	Rol en organización:	Fecha:
Vicente Montesinos Zapata	Técnico en Electrónica	27-07-2026
Revisado por:	Rol en organización:	Fecha:
Aceptado por:	Rol en organización:	Fecha:

Registro de versiones

Versión	Fecha	Secciones afectadas	Autor
V1	27-07-2026		Vicente Montesinos Zapata



Índice

1. Contexto	3
2. Objetivos	3
2.1. Objetivo General	3
2.2. Objetivos Específicos	3
3. Alcances y Limitaciones	3
3.1. Alcances	3
3.2. Limitaciones	4
4. Arquitectura del Sistema	4
4.1. Hardware y Conexiones Físicas	4
4.2. Estructura de Datos (Payload)	5
5. Modo de Uso	5
5.1. Puesta en Marcha del ESP32-S3	5
5.2. Registro de Datos en el Computador	5
6. Pruebas y Mediciones	6
7. Anexo	7
A. Código ESP32-S3	7
B. Código PC conectado a la red	9

	Sistema de Monitoreo de Temperatura para Radiómetro	Monitoreo Temperatura Fecha: 28-08-2026 Estado: En revisión Página: 3 de 10
---	--	--

1. Contexto

El presente reporte documenta la implementación de un sistema de monitoreo de temperaturas para operar en la estructura de una antena y su ambiente. Se busca superar las limitaciones de latencia presentadas por arquitecturas anteriores basadas enteramente en sistemas operativos embebidos (como Raspberry Pi), trasladando la carga de lectura de sensores a un microcontrolador ESP32-S3 y delegando la recepción y almacenamiento de datos a un computador mediante red local.

2. Objetivos

2.1. Objetivo General

Desarrollar y validar una arquitectura de telemetría de baja latencia utilizando sensores DS18B20, procesamiento en ESP32 y transmisión UDP, para registrar valores térmicos en tiempo real.

2.2. Objetivos Específicos

- Implementar la lectura asíncrona de dos sensores DS18B20 utilizando el bus 1-Wire sin bloquear el procesador principal.
- Transmitir estructuras de datos de tamaño reducido (12 bytes) mediante *broadcast* UDP a la red local.
- Desarrollar un script en Python capaz de recibir y registrar los datos en formato CSV (Comma Separated Values) a máxima velocidad.

3. Alcances y Limitaciones

3.1. Alcances

El sistema abarca el diseño del circuito de captura (ESP32 + DS18B20), el código de lectura asíncrona a 9 bits de resolución, y la herramienta de logging en Python. El formato de captura de datos incluye el timestamp relativo del microcontrolador y la lectura precisa

en grados Celsius de los dos sensores térmicos, almacenados localmente para su posterior análisis.

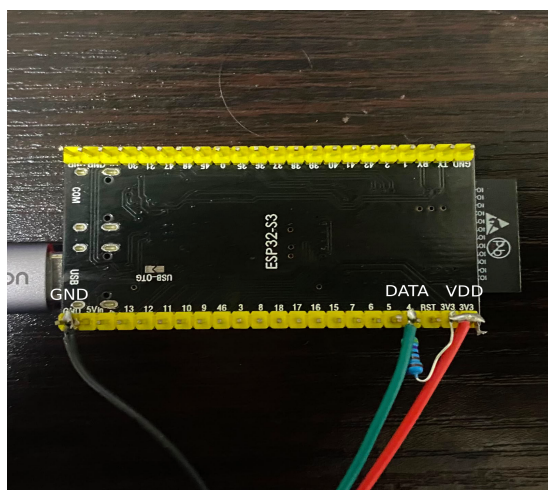
3.2. Limitaciones

A pesar del requerimiento inicial de lograr un muestreo cada 10 ms (100 Hz), el sistema está físicamente limitado por el protocolo 1-Wire y el tiempo de conversión analógico a digital del sensor DS18B20. A su resolución mínima de 9 bits, el sensor requiere un tiempo garantizado de 94 ms.

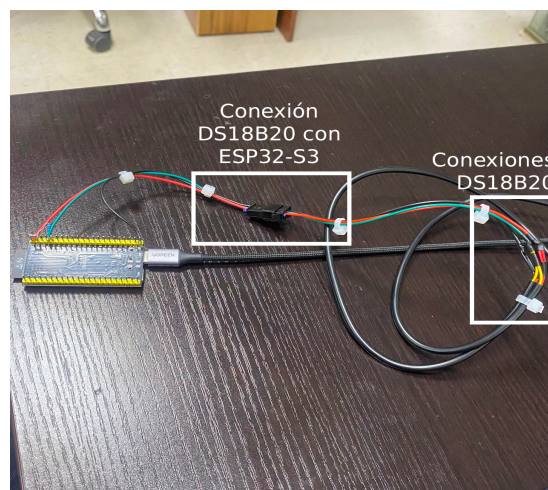
4. Arquitectura del Sistema

4.1. Hardware y Conexiones Físicas

Se utilizó un bus 1-Wire configurado en paralelo, permitiendo la conexión de los sensores de antenna y ambiente al mismo pin de datos (GPIO 4) del ESP32. Para estabilizar el bus se implementó una resistencia *pull-up* de 4.7 k Ω entre la línea de 3.3V (VDD) y DATA.



(a) Detalle de soldadura y resistencia *pull-up*.



(b) Conexión al microcontrolador ESP32.

Figura 1: Montaje del hardware.

	Sistema de Monitoreo de Temperatura para Radiómetro	Monitoreo Temperatura Fecha: 28-08-2026 Estado: En revisión Página: 5 de 10
---	---	--

4.2. Estructura de Datos (Payload)

Para maximizar la eficiencia, los datos se empaquetan en una estructura (C++ *struct*) empaquetada (`__attribute__((packed))`) de 12 bytes:

- `uint32_t` timestamp_ms (4 bytes)
- `float` temp_ambiente (4 bytes)
- `float` temp_antena (4 bytes)

5. Modo de Uso

El funcionamiento del sistema se divide en dos etapas: la puesta en marcha del microcontrolador y la ejecución del programa receptor en el computador.

5.1. Puesta en Marcha del ESP32-S3

Alimentar el ESP32-S3 mediante el puerto USB. Durante el arranque, el dispositivo se conectará a la red WiFi (configurado para red *TESTBED*), detectará automáticamente los sensores DS18B20 presentes en el bus 1-Wire y comenzará a transmitir continuamente los datos mediante paquetes UDP por la red.

5.2. Registro de Datos en el Computador

Conectar el computador a la misma red local que utiliza el ESP32-S3 (red WiFi *TESTBED*, contraseña: `cepia2022`). Copiar el script de recepción mostrado en el *Apéndice B* al computador receptor. Abrir una terminal en la carpeta donde se encuentra el script y ejecutar:

```
python3 logger_udp.py
```

El programa comenzará inmediatamente a escuchar los paquetes UDP enviados por el ESP32-S3. Cada paquete recibido contiene:

- La marca de tiempo (*timestamp*).

	Sistema de Monitoreo de Temperatura para Radiómetro	Monitoreo Temperatura Fecha: 28-08-2026 Estado: En revisión Página: 6 de 10
---	---	--

- La temperatura del sensor de ambiente.
- La temperatura del sensor instalado en la antena.

Los datos serán almacenados automáticamente en un archivo con nombre similar a:

`registro_temperaturas_AñoMesDia_HoraMinutoSegundo.csv`

Cada fila del archivo CSV corresponde a una medición y contiene las siguientes columnas:

- Timestamp.
- Tiempo transcurrido desde el inicio del registro.
- Temperatura ambiente.
- Temperatura de la antena.

Una vez iniciado el programa receptor, el registro continuará de forma indefinida hasta que el usuario detenga la ejecución del script (la ejecución se detiene con: Ctrl+C).

6. Pruebas y Mediciones

Las pruebas revelaron una base sólida de lectura. Al procesar el archivo resultante, se determinó que la latencia física real del sistema implementado es de **165 ms** (aproximadamente 6 Hz).

Este valor de 165 ms se desglosa internamente de la siguiente manera:

1. ~15 ms para emitir los comandos *Skip ROM* y *Convert T*.
2. 95 ms dedicados a la conversión ADC a 9 bits (asíncrona).
3. ~55 ms requeridos para extraer secuencialmente el *scratchpad* de ambos sensores en el bus.

Durante las pruebas se evidenciaron intervalos atípicos de 330 ms y 495 ms. Estos responden a múltiplos exactos de la latencia base (165×2 y 165×3), demostrando que la máquina de estados del ESP32 operó sin bloqueos, pero ocurrieron pérdidas ocasionales de datagramas UDP en la capa de red WiFi.

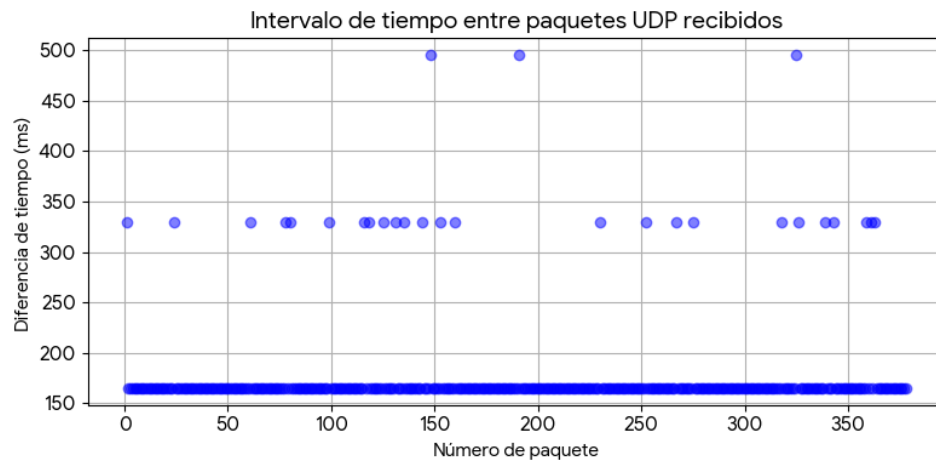


Figura 2: Gráfica de intervalos de recepción UDP, mostrando la moda en 165 ms y saltos esporádicos por pérdida de paquetes.

7. Anexo

A. Código ESP32-S3

Visualizar código en este link.

```
#include <WiFi.h>
#include <WiFiUdp.h>
#include <OneWire.h>
#include <DallasTemperature.h>

// Configuración de Red
const char* ssid = "TESTBED";
const char* password = "cepia2022";
const char* udpAddress = "255.255.255.255";
const int udpPort = 12345;

// Configuración DS18B20
const int oneWireBus = 4; // Pin de datos de los sensores
OneWire oneWire(oneWireBus);
DallasTemperature sensors(&oneWire);

WiFiUDP udp;
```



```
// Estructura de datos (empaquetada para evitar bytes de relleno/padding)
struct __attribute__((packed)) SensorData {
    uint32_t timestamp_ms; // 4 bytes
    float temp_ambiente;    // 4 bytes
    float temp_antena;      // 4 bytes
};
SensorData dataPacket;

// Variables de control de tiempo
unsigned long lastRequest = 0;
const int conversionTime = 95; // ~94ms necesarios para 9 bits, redondeado a 95ms
bool requestPending = false;

void setup() {
    Serial.begin(115200);

    // Iniciar WiFi
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("\nWiFi conectado. IP: " + WiFi.localIP().toString());

    // Iniciar sensores
    sensors.begin();
    sensors.setResolution(9); // Resolución de 9 bits (0.5°C), permite lecturas cada ~94ms

    // CRÍTICO: Desactiva el bloqueo. requestTemperatures() retornará instantáneamente
    sensors.waitForConversion(false);
}

void loop() {
    unsigned long currentMillis = millis();

    // Paso 1: Si no hay una petición pendiente, ordenamos a los sensores que midan
    if (!requestPending) {
        sensors.requestTemperatures();
        lastRequest = currentMillis;
        requestPending = true;
    }
    // Paso 2: Si ya pedimos la medición, revisamos si ya pasó el tiempo necesario (95ms)
```

```
else {
    if (currentMillis - lastRequest >= conversionTime) {
        // Poblar la estructura
        dataPacket.timestamp_ms = currentMillis;
        dataPacket.temp_ambiente = sensors.getTempCByIndex(0);
        dataPacket.temp_antena = sensors.getTempCByIndex(1);

        // Enviar por UDP
        udp.beginPacket(udpAddress, udpPort);
        udp.write((uint8_t*)&dataPacket, sizeof(dataPacket));
        udp.endPacket();

        // Reiniciamos la bandera para pedir una nueva medición en el próximo ciclo
        requestPending = false;
    }
}
}
```

B. Código PC conectado a la red

Visualizar código en este link.

```
import socket
import struct
import datetime
import csv

UDP_IP = "0.0.0.0"
UDP_PORT = 12345

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind((UDP_IP, UDP_PORT))

print(f"Escuchando datos UDP en el puerto {UDP_PORT}...")

tiempo_inicio = datetime.datetime.now()
str_inicio = tiempo_inicio.strftime("%Y%m%d_%H%M%S")
nombre_archivo = f"registro_temperaturas_{str_inicio}.csv"
```



```
print(f"Los datos se guardarán en el nuevo archivo: {nombre_archivo}\n")

offset_ms = None
with open(nombre_archivo, mode='w', newline='') as archivo_csv:
    escritor_csv = csv.writer(archivo_csv)
    escritor_csv.writerow(["Hora Local", "Tiempo Relativo (ms)", "T. Ambiente (C)", ""])

    try:
        while True:
            data, addr = sock.recvfrom(1024)

            ahora = datetime.datetime.now()
            hora_formateada = ahora.strftime("%H:%M:%S.%f")[:-3]

            timestamp_esp32_crudo, temp_amb, temp_ant = struct.unpack('<Iff', data)

            if offset_ms is None:
                offset_ms = timestamp_esp32_crudo
                print(f"Sincronizando tiempo... Offset inicial guardado: {offset_ms} ms")

            tiempo_relativo = timestamp_esp32_crudo - offset_ms

            escritor_csv.writerow([hora_formateada, tiempo_relativo, f"{temp_amb:.2f}"])
            print(f"[{hora_formateada}] t= {tiempo_relativo} ms | Ambiente: {temp_amb:.2f} C")

    except KeyboardInterrupt:
        print(f"\nCaptura finalizada de forma segura.")
        print(f"El archivo {nombre_archivo} se ha cerrado y guardado correctamente.")
        sock.close()
```